EPI Quantum Crypto team
Cryptographie appliquée et évaluation des sources d'entropie

GUIDE D'UTILISATION · VERSION 3.1

libsp80090b : API C, labs et code d'intégration

Compiler la bibliothèque, appeler ses quatre fonctions, générer les données et lancer les labs, intégrer une source dans un générateur hybride

RÉFÉRENCE	NT-02
OBJET	libsp80090b : mode d'emploi de la bibliothèque et des labs
VERSION	3.1 — septembre 2026
AUTEURS	EPI Quantum Crypto team
DIFFUSION	Guide d'utilisation — diffusion publique
RÉFÉRENTIELS	NIST SP 800-90B, SP 800-90C ; FIPS 140-3 (IG D.K)
DOCUMENTS LIÉS	NT-01 Note technique (méthode) ; NT-03 Guide de bonnes pratiques ; version anglaise : NT-02 (en)
REMPLACE	NT-02 v3.0

RÉSUMÉ

Ce guide s'adresse aux développeurs C, aux ingénieurs d'intégration et aux opérateurs de banc d'essai. La note NT-01 établit la méthode de NIST SP 800-90B ; ce guide donne le mode d'emploi opérationnel de **libsp80090b** : la compilation et ses prérequis (C99, OpenMP, libdivsufsort, libbz2), les structures et les quatre fonctions de l'API C, la génération des données et les quatre labs, et le code C d'intégration dans un générateur hybride : qualification sur la machine cible, tests de santé RCT et APT en continu, collecte de la graine du DRBG. Les résultats chiffrés et les limites connues sont dans le rapport NT-03.

Mots-clés : libsp80090b ; API C ; sp80090b_iid_evaluate_ex ; sp80090b_non_iid_evaluate_ex ; sp80090b_restart_evaluate ; sp80090b_conditioning_evaluate ; gen_data.sh ; labs ; tests de santé ; graine

Sommaire

1 Présentation et arborescence 3

2 Compilation et prérequis 3

2.1 Dépendances 3

2.2 Cibles du Makefile 3

3 L'API C 4

3.1 Options et résultat des tests 1 et 2 4

3.2 Les quatre fonctions 4

3.3 Codes de retour et avertissements 5

4 Données et labs 6

4.1 Générer les données : lab/gen_data.sh 6

4.2 Enregistrer du vrai bruit : jitterentropy 6

4.3 Lancer les quatre labs 6

5 Code C d'intégration 7

5.1 Décision de conformité et qualification 7

5.2 Tests de santé RCT et APT en continu 8

5.3 Démarrage et collecte de la graine du DRBG 9

6 Vérifier une installation 9

7 Recommandations d'intégration 10

8 Références 10

1 Présentation et arborescence

libsp80090b est un portage en C99 de l'outil du NIST SP800-90B_EntropyAssessment 1.1.8 [2]. Elle évalue un **tampon en mémoire** : aucun fichier à écrire, aucun exécutable à lancer, aucune sortie texte à analyser.

Arborescence de SP800-90B_lib

```
SP800-90B_lib/
├── Makefile                  make, make lib, make lab, make check, make docs
├── README.md, README.fr.md   entry point (English, French)
├── include/
│   ├── sp80090b.h            public API: the only header to include
│   └── *.h                   internal headers, one per estimator (ported from the NIST C++
code)
├── src/
│   ├── sp80090b_api.c        the four API functions
│   └── test_*.c              test_iid, test_non_iid, test_restart, test_conditioning,
test_selftest
├── lab/
│   ├── gen_data.sh           generator of the 24 lab datasets
│   ├── lab1.sh .. lab4.sh    the four labs
│   ├── data/                 lab1_iid/, lab2_non_iid/, lab3_restart/, lab4_conditioning/
│   ├── results/              same directories: <case>.res (full output), labN.log
│   ├── lab_*.c               command-line tools compatible with the NIST ea_* tools
│   └── jent_record.c         recorder of jitterentropy raw noise
├── bin/                      the 11 NIST reference datasets
├── selftest/                 copy of the NIST selftest (conformity, section 6)
└── docs/                     NT-01, NT-02, NT-03: sources, style, publications
```

2 Compilation et prérequis

2.1 Dépendances

Paquets Ubuntu / Debian

```
sudo apt-get install -y build-essential libdivsufsort-dev libbz2-dev openssl
```

La bibliothèque ne dépend d'aucune bibliothèque C++, ni de jsoncpp, gmp ou mpfr. À l'exécution, elle n'utilise que libdivsufsort [8] (tableau des suffixes, pour le test de la plus longue répétition), libbz2 (statistique de compression), libgomp (parallélisation OpenMP), libm et libc. openssl ne sert qu'à `gen_data.sh`.

2.2 Cibles du Makefile

Cibles du Makefile

```
make            # static and shared libraries, labs and test programs
make lib        # libsp80090b.a and libsp80090b.so only
make lab        # lab/lab_iid, lab_non_iid, lab_restart, lab_conditioning
make check      # conformity checks (section 6)
make clean      # remove binaries and objects
```

Optimisation et portabilité. Par défaut, `CFLAGS` vaut `-std=c99 -O2 -fopenmp -ffloat-store -ffp-contract=fast -march=native`. L'option `-ffp-contract=fast` contracte les multiplications-additions comme le compilateur C++ du NIST : c'est ce qui rend les résultats identiques au bit près à ceux des outils `ea_*` compilés sur la même machine. Le binaire ne tourne alors que sur des processeurs équivalents. Pour un binaire portable, fixer l'architecture avec `make MARCH=-march=x86-64-v2` ; les derniers chiffres des résultats peuvent alors différer.

3 L'API C

Un seul en-tête est à inclure, `sp80090b.h`, et la liaison se fait ainsi :

Compilation et liaison

```
gcc -std=c99 -O2 -fopenmp main.c -I./include libsp80090b.a \
    -ldivsufsort -ldivsufsort64 -lbz2 -lm -o my_app
```

3.1 Options et résultat des tests 1 et 2

sp80090b.h : options et résultat

```
typedef struct {
    int bits_per_symbol;       /* 1..8, or 0 to infer from the data */
    bool initial_entropy;      /* true: -i, H_I of raw noise; false: -c, h' of conditioned data
*/
    bool truncate_bitstring;   /* true: -t, H_bitstring on the first 1,000,000 bits */
    int verbose;               /* 0 silent; 1..3 as the NIST tool, on stdout */
    const uint64_t *seed;      /* IID only: 4 words for reproducible permutations; NULL =
/dev/urandom */
} sp80090b_options_t;

typedef struct {
    double h_original;         /* min-entropy of the samples, bits per sample */
    double h_bitstring;        /* min-entropy of the bit string, bits per bit; h' with -c */
    double h_assessed;         /* min(H_original, n x H_bitstring); h' with -c */
    bool passed_iid;           /* IID: chi-square AND LRS AND permutation tests passed */
    double p_value;            /* IID: smaller chi-square p-value */
    int status;                /* return code of the call */
    char error_msg[256];       /* error text, or last warning text */
    unsigned int warnings;     /* SP80090B_WARN_* flags */
    long n_samples;            /* samples assessed */
    int bits_per_symbol;       /* width used (inferred when 0 was given) */
    int alphabet_size;         /* distinct symbols observed */
    bool passed_chi_square;    /* IID: 5.2.1-5.2.4 */
    bool passed_lrs;           /* IID: 5.2.5 */
    bool passed_permutation;   /* IID: 5.1 */
    int perm_counts[19][3];    /* IID: C[i][0] (>), C[i][1] (=), C[i][2] (<) for the 19
statistics */
} sp80090b_result_t;
```

3.2 Les quatre fonctions

Test 1 : voie IID (§5).

Test 1

```
int sp80090b_iid_evaluate_ex(const uint8_t *data, size_t len,
                             const sp80090b_options_t *opt, sp80090b_result_t *result);
```

> **Attention.** `result->h_assessed` n'est utilisable **que si** `result->passed_iid` **est vrai**. Sinon, la source doit être évaluée par la voie non IID.

Test 2 : voie non IID (§6.3). Calcule les dix estimateurs et renvoie H_I dans `result->h_assessed`.

Test 2

```
int sp80090b_non_iid_evaluate_ex(const uint8_t *data, size_t len,
                                 const sp80090b_options_t *opt, sp80090b_result_t *result);
```

Test 3 : redémarrage (§3.1.4).

Test 3

```
int sp80090b_restart_evaluate(const uint8_t *data, size_t len, int bits_per_symbol,
                              double h_i, bool iid, unsigned long rounds,
                              const uint64_t *seed, int verbose,
                              sp80090b_restart_result_t *result);
```

- `data` : matrice de 1 000 × 1 000 échantillons rangée par lignes, le redémarrage i à l'offset 1 000 × i ; `len` vaut 1 000 000.
- `h_i` : H_I issu du test 1 ou du test 2 ; `iid` choisit la voie IID sur les lignes et les colonnes.
- `rounds` : tirages de la simulation du seuil ; 0 pour les 5 000 000 du NIST, une valeur inférieure est refusée.
- Résultat : `sanity_passed` (§3.1.4.3), `validation_passed` (§3.1.4.2), `x_max`, `x_cutoff`, `h_r`, `h_c` et l'entropie validée `h_restart` = min(H_r, H_c, H_I).

Test 4 : conditionnement (§3.1.5).

Test 4

```
int sp80090b_conditioning_evaluate(bool vetted, unsigned int n_in, unsigned int n_out,
                                   unsigned int nw, long double h_in, long double h_prime,
                                   sp80090b_conditioning_result_t *result);
```

- `vetted` : vrai pour une fonction approuvée (SHA-2, SHA-3, HMAC, CMAC, CBC-MAC) ; sinon `h_prime` ∈]0, 1] est l'entropie par bit de la sortie, mesurée par la voie non IID en mode `-c`.
- `h_in` : entropie totale d'entrée, nombre d'échantillons × H.
- Résultat : `h_out`, `epsilon_log2` et `full_entropy_candidate`, vrai si la fonction est approuvée, si $h_{out} > 0{,}999\ n_{out}$ et si $h_{in} \geq n_{out} + 64$ (FIPS 140-3 IG D.K). C'est la dernière condition qui décide : avec une fonction approuvée et une marge de 64 bits, le déficit d'entropie par bit est inférieur à 2^{-32}.

3.3 Codes de retour et avertissements

Tableau 1. Codes de retour.

Code	Valeur	Signification
`SP80090B_OK`	0	Évaluation faite ; lire ensuite `warnings`
`SP80090B_EINVAL`	−1	Argument invalide : pointeur nul, moins de 2 échantillons, largeur hors de [0, 8], paramètres de redémarrage ou de conditionnement
`SP80090B_EDATA`	−2	Données plus larges que `bits_per_symbol`, ou mémoire insuffisante à la lecture
`SP80090B_EALPHABET`	−3	**Source morte** : un seul symbole dans tout le tampon
`SP80090B_EINTERNAL`	−4	Échec d'allocation, de graine (`/dev/urandom`) ou numérique

Tableau 2. Drapeaux d'avertissement (champ warnings).

Drapeau	Masque	Signification
`SP80090B_WARN_FEW_SAMPLES`	0x1	Moins de 1 000 000 d'échantillons (§3.1.1)
`SP80090B_WARN_NARROW_DATA`	0x2	Symboles plus étroits que la largeur déclarée
`SP80090B_WARN_TRUNCATED`	0x4	$H_{bitstring}$ calculé sur le premier million de bits seulement (`-t`)

Un avertissement n'est pas une erreur, mais il invalide un résultat destiné à qualifier une source.

4 Données et labs

Les outils du NIST et la bibliothèque attendent un fichier binaire, **un échantillon par octet** : pour une source de 1 bit, chaque octet vaut 0 ou 1.

4.1 Générer les données : lab/gen_data.sh

`gen_data.sh` construit 24 jeux de données en 7 groupes : sources physiques **simulées** (bruit thermique, oscillateur en anneau…) tirées d'un modèle à graine, défauts de conception, effets d'environnement, pannes, sorties de DRBG issues du système (`/dev/urandom`, `openssl`) et jeux de redémarrage. Chaque fichier est copié dans le répertoire de chaque lab qui le lit.

Générer les données

```
lab/gen_data.sh all --seed 90       # the 24 datasets, fixed seed
lab/gen_data.sh thermal --sd 5      # one dataset, with a model parameter changed
lab/gen_data.sh --list              # groups, datasets and what each one shows
```

4.2 Enregistrer du vrai bruit : jitterentropy

`lab/jent_record` enregistre le bruit brut de gigue du processeur mesuré par jitterentropy [9], en `-O0` comme la bibliothèque elle-même ; la variable `JENT_SRC` indique ses sources.

Enregistrer le bruit brut de jitterentropy

```
make lab/jent_record
lab/jent_record lab/data/lab2_non_iid/jent-raw.bin          # 1,000,000 samples (tests 1 and 2)
lab/jent_record -r lab/data/lab3_restart/jent-restart.bin   # 1,000 restarts x 1,000 (test 3)
```

4.3 Lancer les quatre labs

Chaque lab parcourt les fichiers de son répertoire de données, écrit la sortie complète dans `lab/results/<lab>/<cas>.res` et affiche une ligne par cas. Les labs se lancent un par un, dans cet ordre :

Lancer les labs

```
lab/lab2.sh     # non-IID track on lab2_non_iid/: H_I, needed by labs 3 and 4
lab/lab3.sh     # restart test on lab3_restart/
lab/lab4.sh     # conditioning cases
lab/lab1.sh     # IID track on lab1_iid/: several hours if sources are not IID
```

Tableau 3. Les quatre labs.

Lab	Programme	Données	Durée par fichier de 10^6 échantillons
1	`test_iid -vv`	`lab1_iid/`	Moins d'une minute si IID ; 40 minutes à 2 heures sinon
2	`test_non_iid -vv`	`lab2_non_iid/`	5 secondes à 1 minute
3	`test_restart -v`	`lab3_restart/`	40 secondes à 1 minute 30
4	`test_conditioning`	`lab4_conditioning/`	Moins d'une seconde par cas

Un fichier se lance aussi seul, depuis `lab/`, par exemple pour reprendre un lab interrompu : `../test_iid -vv data/lab1_iid/drift.bin 8 > results/lab1_iid/drift.res 2>&1`. Les programmes `lab/lab_*` reprennent en plus la ligne de commande des outils `ea_*` du NIST, pour comparer un résultat.

5 Code C d'intégration

Cette section donne un exemple de code C pour un générateur hybride (SP 800-90C) : décision de conformité à la qualification, puis tests de santé et collecte de la graine en marche.

5.1 Décision de conformité et qualification

La fonction applique les critères de NT-01 : H_I par la voie non IID, test de redémarrage complet, puis comparaison avec la revendication du modèle.

Décision de conformité d'une source

```
#include <stdbool.h>
#include <stdint.h>
#include "sp80090b.h"

typedef enum {
    ENTROPY_PASSED            =  0,   /* conforming source */
    ERR_INSUFFICIENT_SAMPLES  = -1,   /* fewer than 1,000,000 samples */
    ERR_SOURCE_STUCK          = -2,   /* a single symbol: dead source */
    ERR_RESTART_SANITY_FAILED = -3,   /* X_max > X_cutoff (3.1.4.3) */
    ERR_RESTART_ENTROPY_DROP  = -4,   /* min(H_r, H_c) < H_I / 2 (3.1.4.2) */
    ERR_ENTROPY_BELOW_CLAIM   = -5,   /* validated H below the claim */
    ERR_EVALUATION            = -6    /* argument, data or memory error */
} entropy_decision_t;

/*
 * raw     : at least 1,000,000 consecutive raw samples (steady state)
 * rows    : 1000 restarts x 1000 samples, restart i at offset 1000 * i
 * h_claim : entropy per sample claimed by the model of the source
 * h_out   : credited entropy, min(H_r, H_c, H_I, h_claim)
 */
entropy_decision_t verify_entropy_compliance(const uint8_t *raw, size_t len,
                                             const uint8_t *rows, int bits,
                                             double h_claim, double *h_out)
{
    sp80090b_options_t opt = sp80090b_default_options();
    sp80090b_result_t res;
    sp80090b_restart_result_t rst;
    double h_i, h_measured;
    int status;

    if (!raw || !rows || !h_out)
        return ERR_EVALUATION;

    /* Steps 1-2: H_I through the non-IID track */
    opt.bits_per_symbol = bits;
    status = sp80090b_non_iid_evaluate_ex(raw, len, &opt, &res);
    if (status == SP80090B_EALPHABET)
        return ERR_SOURCE_STUCK;
    if (status != SP80090B_OK)
        return ERR_EVALUATION;
    if (res.warnings & SP80090B_WARN_FEW_SAMPLES)
        return ERR_INSUFFICIENT_SAMPLES;
    if (res.warnings != 0)                    /* any warning invalidates a qualification */
        return ERR_EVALUATION;

    /* SP 800-90B: H_I includes the submitter's claim */
    h_i = res.h_assessed < h_claim ? res.h_assessed : h_claim;

    /* Steps 3-4: restart test, sanity check then H_r and H_c */
    status = sp80090b_restart_evaluate(rows, (size_t)SP80090B_RESTART_ROWS *
SP80090B_RESTART_COLS,
                                       bits, h_i, false, 0, NULL, 0, &rst);
    if (status != SP80090B_OK)
        return ERR_EVALUATION;
    if (!rst.sanity_passed)
```

Décision de conformité d'une source

```
        return ERR_RESTART_SANITY_FAILED;
    if (!rst.validation_passed)
        return ERR_RESTART_ENTROPY_DROP;

    /* Step 5: credit the validated entropy; flag a measurement below the claim */
    *h_out = rst.h_restart;                     /* = min(H_r, H_c, H_I) <= h_claim */
    h_measured = res.h_assessed;
    if (rst.h_r < h_measured) h_measured = rst.h_r;
    if (rst.h_c < h_measured) h_measured = rst.h_c;
    return h_measured >= h_claim ? ENTROPY_PASSED : ERR_ENTROPY_BELOW_CLAIM;
}
```

5.2 Tests de santé RCT et APT en continu

Ce module léger s'exécute dans le générateur sur chaque échantillon produit. Ses seuils se calculent une fois, à partir de l'entropie créditée H ($\alpha = 2^{-20}$) ; ils retrouvent ceux de NT-01.

Tests de santé en continu (SP 800-90B 4.4)

```
#include <math.h>
#include <stdbool.h>
#include <stdint.h>

typedef struct {
    unsigned int rct_c;     /* RCT cutoff */
    unsigned int apt_c;     /* APT cutoff */
    unsigned int apt_w;     /* APT window: 1024 for a binary source, 512 otherwise (4.4.2) */
    uint8_t rct_last;       /* last value seen by the RCT */
    uint8_t apt_base;       /* value tracked in the current APT window */
    unsigned int rct_cnt;   /* RCT counter */
    unsigned int apt_cnt;   /* APT counter */
    unsigned int apt_idx;   /* position in the APT window */
} health_ctx_t;

/* APT cutoff: smallest C such that P(X >= C) <= 2^-20, X ~ Binomial(w, 2^-h) */
static unsigned int compute_apt_cutoff(unsigned int w, double h)
{
    double p = exp2(-h), tail = 0.0;

    for (unsigned int c = w; c > 0; c--) {
        tail += exp(lgamma(w + 1.0) - lgamma(c + 1.0) - lgamma(w - c + 1.0)
                    + c * log(p) + (w - c) * log1p(-p));
        if (tail > 0x1p-20)
            return c + 1;
    }
    return 1;
}

/* Cutoffs from the credited entropy H */
void health_init(health_ctx_t *ctx, double h_credited, bool is_binary)
{
    ctx->rct_c = 1 + (unsigned int)ceil(20.0 / h_credited);
    ctx->apt_w = is_binary ? 1024 : 512;
    ctx->apt_c = compute_apt_cutoff(ctx->apt_w, h_credited);
    ctx->rct_cnt = 0;
    ctx->apt_idx = 0;
}

/* 0 if the sample passes, -1 if the source must be stopped */
int health_check_sample(health_ctx_t *ctx, uint8_t sample)
{
    if (ctx->rct_cnt > 0 && sample == ctx->rct_last) {
        if (++ctx->rct_cnt >= ctx->rct_c)
            return -1;                          /* RCT: abnormal repetition */
    } else {
```

Tests de santé en continu (SP 800-90B 4.4)

```
        ctx->rct_last = sample;
        ctx->rct_cnt = 1;
    }
    if (ctx->apt_idx == 0) {                    /* start of an APT window */
        ctx->apt_base = sample;
        ctx->apt_cnt = 1;
    } else if (sample == ctx->apt_base && ++ctx->apt_cnt >= ctx->apt_c) {
        return -1;                              /* APT: abnormal proportion */
    }
    ctx->apt_idx = (ctx->apt_idx + 1) % ctx->apt_w;
    return 0;
}
```

5.3 Démarrage et collecte de la graine du DRBG

Démarrage et collecte de la graine

```
#include <stddef.h>
#include <stdint.h>

uint8_t read_hardware_noise_sample(void);   /* acquisition, specific to the source */

/* Start-up tests: at least 1,024 consecutive samples must pass (4.3) */
int health_startup(health_ctx_t *ctx)
{
    for (unsigned int i = 0; i < 1024; i++)
        if (health_check_sample(ctx, read_hardware_noise_sample()) != 0)
            return -1;
    return 0;
}

/* Wipe that the compiler cannot remove (use explicit_bzero or memset_s when available) */
static void secure_wipe(void *p, size_t n)
{
    volatile uint8_t *v = (volatile uint8_t *)p;
    while (n--)
        *v++ = 0;
}

/* Seed: N = ceil(E / H) raw samples, all tested; at the first failure, the seed is wiped */
int collect_drbg_seed(health_ctx_t *ctx, uint8_t *seed_buf, size_t n_samples)
{
    for (size_t i = 0; i < n_samples; i++) {
        uint8_t raw = read_hardware_noise_sample();

        if (health_check_sample(ctx, raw) != 0) {
            secure_wipe(seed_buf, n_samples);   /* discard the current seed */
            return -1;
        }
        seed_buf[i] = raw;
    }
    return 0;                                   /* entropy input of the DRBG */
}
```

Les N octets de la graine forment l'entrée d'entropie du DRBG, directement ou après conditionnement. Pour une force de 256 bits, N = ⌈384 / H⌉ à l'instanciation et ⌈256 / H⌉ au réensemencement (NT-01).

6 Vérifier une installation

Refaire ces contrôles après chaque changement de compilateur, d'options de compilation ou de machine :

Vérification

```
cd Entropy/SP800-90B_lib
make check
```

`make check` enchaîne deux contrôles :

1. `selftest/selftest`, copie du selftest de l'outil du NIST : il évalue les onze jeux de référence de `bin/` avec `test_non_iid` et compare toutes les valeurs intermédiaires avec les sorties du NIST de `selftest/refdata/`. La vérification est réussie si aucun écart ne dépasse 10^{-10} ; l'écart maximal observé est de $7{,}2 \times 10^{-14}$.
2. `test_selftest` : valeurs du NIST sur des jeux de référence, contrat de l'API (arguments invalides, avertissements, codes de retour), conditionnement et reproductibilité à graine fixée.

7 Recommandations d'intégration

1. **Toujours vérifier le code de retour** (`SP80090B_OK`), puis le champ `warnings`.
2. **Ne jamais utiliser `h_assessed` de la voie IID** si `passed_iid` est faux.
3. **Évaluer le bruit brut seulement**, jamais une sortie de DRBG ou de hachage : elle obtient près de 8 bits quelle que soit sa graine.
4. **Dimensionner la graine** avec $N = \lceil E / H \rceil$: E = 384 bits à l'instanciation, 256 bits au réensemencement.
5. **Exécuter les tests de santé RCT et APT** sur chaque échantillon brut, avant le conditionnement et le DRBG, et les tests de démarrage sur 1 024 échantillons.
6. **Prévoir une marge de 64 bits** d'entropie d'entrée pour toute fonction de conditionnement visant la pleine entropie : $h_{in} \geq n_{out} + 64$.
7. **Limiter les fils OpenMP** (`OMP_NUM_THREADS`) quand plusieurs évaluations tournent en parallèle, et prévoir environ 250 Mo de mémoire par évaluation d'un million d'échantillons.

8 Références

[2] National Institute of Standards and Technology. *SP800-90B_EntropyAssessment*. Code source, github.com/usnistgov/SP800-90B_EntropyAssessment.

[8] Y. Mori. *libdivsufsort — suffix-sorting library*. Code source, version 2.0.

[9] S. Müller. *Jitter RNG (jitterentropy)*. Documentation et code source, version 3.7.1, 2026.

Les numéros de référence sont ceux de la note NT-01.